

Name: Monisha Raghini Chelle

RollNo:2520030537

Section -6

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

18.To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSLAB14  Simple-Commands-Week1  foldername  hello.c  monis  sample1.c
OSLAB                           OSSP-Practical  example.txt             hello       moni     sample1  sample1.c.save
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
OSLAB28  Scheduling-Algorithms  hello  hello.c:Zone.Identifier  hello_link  newfile.txt  test1
Pipes    file1.txt              hello.c  hello:Zone.Identifier    hello_shortcut  skill        virtual_address
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 2 — Create the Signal Behavior Program

Run:

nano signal_behavior.c

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano signal_behavior.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 signal_behavior.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t received_signal = 0;

void signal_handler(int signo)
{
    received_signal = signo;
}

int main(void)
{
    struct sigaction sa;
    sigset_t block_set;
    sigset_t old_set;
    sigset_t pending_set;

    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);

    if (sigaction(SIGINT, &sa, NULL) == -1 ||
        sigaction(SIGUSR1, &sa, NULL) == -1)
    {
        perror("sigaction");
        return 1;
    }

    printf("Q18 - Signal Behavior Program\n");
    printf("PID: %ld\n", (long)getpid());
    printf("Process Group ID: %ld\n", (long)getpgrp());
    printf("Press Ctrl+C to test SIGINT.\n");

    /* Block SIGUSR1 temporarily */
    sigemptyset(&block_set);
    sigaddset(&block_set, SIGUSR1);

    if (sigprocmask(SIG_BLOCK, &block_set, &old_set) == -1)
    {
        perror("sigprocmask");
        return 1;
    }

    printf("\nSIGUSR1 is now blocked.\n");
    printf("Sending SIGUSR1 to this process...\n");

    if (kill(getpid(), SIGUSR1) == -1)
    {
        perror("kill");
        return 1;
    }

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 signal_behavior.c *
printf("PID: %ld\n", (long) getpid());
printf("Process Group ID: %ld\n", (long) getpgrp());
printf("Press Ctrl+C to test SIGINT.\n");

/* Block SIGUSR1 temporarily */
sigemptyset(&block_set);
sigaddset(&block_set, SIGUSR1);

if (sigprocmask(SIG_BLOCK, &block_set, &old_set) == -1)
{
    perror("sigprocmask");
    return 1;
}

printf("\nSIGUSR1 is now blocked.\n");
printf("Sending SIGUSR1 to this process...\n");

if (kill(getpid(), SIGUSR1) == -1)
{
    perror("kill");
    return 1;
}

if (sigpending(&pending_set) == -1)
{
    perror("sigpending");
    return 1;
}

if (sigismember(&pending_set, SIGUSR1))
    printf("SIGUSR1 is pending while blocked.\n");

printf("Unblocking SIGUSR1...\n");

if (sigprocmask(SIG_SETMASK, &old_set, NULL) == -1)
{
    perror("sigprocmask");
    return 1;
}

if (received_signal == SIGUSR1)
    printf("SIGUSR1 delivered successfully.\n");

printf("\nWaiting for terminal signals.\n");
printf("Press Ctrl+C to send SIGINT.\n");
printf("Type Ctrl+C once, then the program will exit.\n");
fflush(stdout);

while (received_signal != SIGINT)
    pause();

printf("\nSIGINT received from terminal.\n");
printf("Signal behavior test completed.\n");

return 0;
}
```

Save with Ctrl + O, Enter, then Ctrl + X.

```
}
Write to File: signal_behavior.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel    M-M Mac Format  M-P Prepend    ^Q Discard buffer
```

Save and compile:

gcc -Wall -Wextra signal_behavior.c -o signal_behavior

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano signal_behavior.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra signal_behavior.c -o signal_behavior
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 3 — Test Signal Delivery and Masks

Run:

./signal_behavior

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./signal_behavior
Q18 - Signal Behavior Program
PID: 884
Process Group ID: 884
Press Ctrl+C to test SIGINT.

SIGUSR1 is now blocked.
Sending SIGUSR1 to this process...
SIGUSR1 is pending while blocked.
Unblocking SIGUSR1...
SIGUSR1 delivered successfully.

Waiting for terminal signals.
Press Ctrl+C to send SIGINT.
Type Ctrl+C once, then the program will exit.
```

Now press Ctrl + C.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill$ ./signal_behavior
Q18 - Signal Behavior Program
PID: 884
Process Group ID: 884
Press Ctrl+C to test SIGINT.

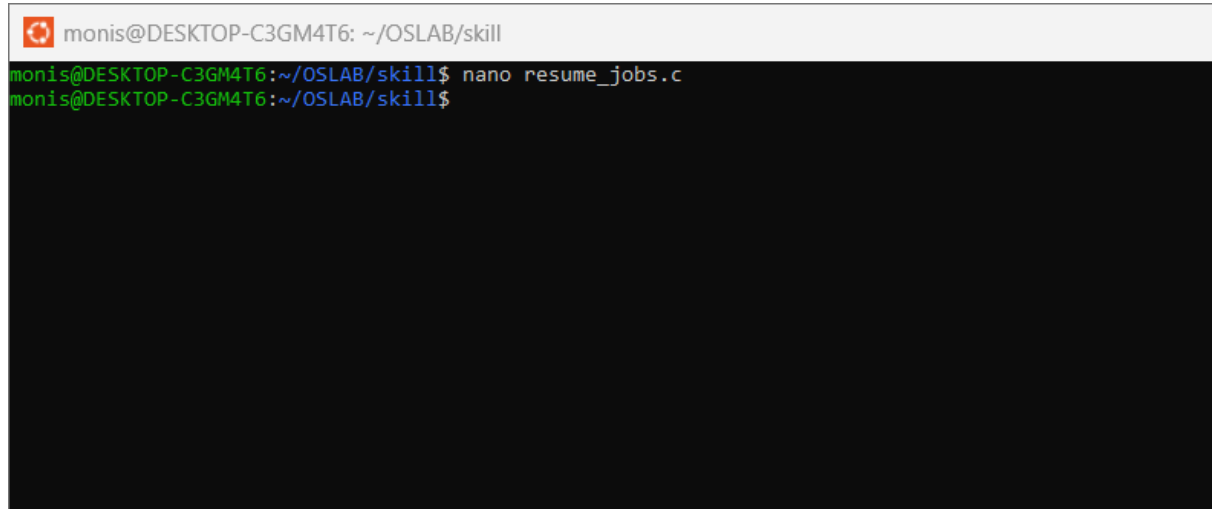
SIGUSR1 is now blocked.
Sending SIGUSR1 to this process...
SIGUSR1 is pending while blocked.
Unblocking SIGUSR1...
SIGUSR1 delivered successfully.

Waiting for terminal signals.
Press Ctrl+C to send SIGINT.
Type Ctrl+C once, then the program will exit.
^]]^]]^]]^]]^]]^]]^]]^]]^]]^C
SIGINT received from terminal.
Signal behavior test completed.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 4 — Create the Background Job Recovery Program

Run:

nano resume_jobs.c



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano resume_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 resume_jobs.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_JOBS 10

typedef enum
{
    EMPTY,
    RUNNING,
    STOPPED,
    COMPLETED
} JobState;

typedef struct
{
    int id;
    pid_t pid;
    pid_t pgid;
    JobState state;
    char command[100];
} Job;

Job jobs[MAX_JOBS];
int next_id = 1;

const char *state_name(JobState state)
{
    switch (state)
    {
        case RUNNING: return "RUNNING";
        case STOPPED: return "STOPPED";
        case COMPLETED: return "COMPLETED";
        default: return "EMPTY";
    }
}

void update_jobs(void)
{
    int status;

    for (int i = 0; i < MAX_JOBS; i++)
    {
        if (jobs[i].state != RUNNING &&
            jobs[i].state != STOPPED)
            continue;

        pid_t result;

        do
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
do
{
    result = waitpid(
        jobs[i].pid,
        &status,
        WNOHANG | WUNTRACED | WCONTINUED
    );
}
while (result == -1 && errno == EINTR);

if (result == jobs[i].pid)
{
    if (WIFSTOPPED(status))
        jobs[i].state = STOPPED;

    else if (WIFCONTINUED(status))
        jobs[i].state = RUNNING;

    else if (WIFEXITED(status) ||
             WIFSIGNALED(status))
        jobs[i].state = COMPLETED;
}
}

void list_jobs(void)
{
    update_jobs();

    printf("\nJOB TABLE\n");
    printf("ID\tPID\tPGID\tSTATE\tCOMMAND\n");

    int found = 0;

    for (int i = 0; i < MAX_JOBS; i++)
    {
        if (jobs[i].state != EMPTY)
        {
            printf(
                "%d\t%d\t%d\t%-10s\t%s\n",
                jobs[i].id,
                (long)jobs[i].pid,
                (long)jobs[i].pgid,
                state_name(jobs[i].state),
                jobs[i].command
            );

            found = 1;
        }
    }

    if (!found)
        printf("No jobs found.\n");
}

int find_job(int id)
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 resume_jobs.c *
int find_job(int id)
{
    for (int i = 0; i < MAX_JOBS; i++)
    {
        if (jobs[i].state != EMPTY &&
            jobs[i].id == id)
            return i;
    }
    return -1;
}

void start_job(int seconds)
{
    update_jobs();

    int slot = -1;

    for (int i = 0; i < MAX_JOBS; i++)
    {
        if (jobs[i].state == EMPTY ||
            jobs[i].state == COMPLETED)
        {
            slot = i;
            break;
        }
    }

    if (slot == -1)
    {
        printf("Job table is full.\n");
        return;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        if (setpgid(0, 0) == -1)
        {
            perror("setpgid");
            _exit(1);
        }

        char duration[20];

        snprintf(duration, sizeof(duration),
                 "%d", seconds);

        execlp("sleep", "sleep",
               duration, (char *)NULL);
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^I Paste ^J Justify ^_ Go To Line M-F Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 resume_jobs.c *
    execlp("sleep", "sleep",
          duration, (char *)NULL);

    perror("execlp");
    _exit(127);
}

if (setpgid(pid, pid) == -1 &&
    errno != EACCES)
{
    perror("setpgid");
}

jobs[slot].id = next_id++;
jobs[slot].pid = pid;
jobs[slot].pgid = pid;
jobs[slot].state = RUNNING;

snprintf(
    jobs[slot].command,
    sizeof(jobs[slot].command),
    "sleep %d",
    seconds
);

printf(
    "Started job [%d], PID=%ld, PGID=%ld\n",
    jobs[slot].id,
    (long)pid,
    (long)pid
);
}

void stop_job(int id)
{
    update_jobs();

    int index = find_job(id);

    if (index == -1 ||
        jobs[index].state != RUNNING)
    {
        printf("Running job not found.\n");
        return;
    }

    if (kill(-jobs[index].pgid, SIGSTOP) == -1)
    {
        perror("SIGSTOP");
        return;
    }

    int status;

    if (waitpid(jobs[index].pid,
                &status, WUNTRACED) == -1)
    {
        ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
        ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
    }
}
```

```
        perror("waitpid");
        return;
    }

    if (WIFSTOPPED(status))
    {
        jobs[index].state = STOPPED;
        printf("Job [%d] stopped.\n", id);
    }
}

void resume_job(int id)
{
    update_jobs();

    int index = find_job(id);

    if (index == -1 ||
        jobs[index].state != STOPPED)
    {
        printf("Stopped job not found.\n");
        return;
    }

    if (kill(-jobs[index].pgid, SIGCONT) == -1)
    {
        perror("SIGCONT");
        return;
    }

    jobs[index].state = RUNNING;

    printf(
        "SIGCONT sent to job [%d].\n",
        id
    );

    printf("Job resumed in background.\n");
}

void remove_completed(void)
{
    update_jobs();

    int removed = 0;

    for (int i = 0; i < MAX_JOBS; i++)
    {
        if (jobs[i].state == COMPLETED)
        {
            printf(
                "Removing completed job [%d].\n",
                jobs[i].id
            );

            memset(&jobs[i], 0,
                sizeof(jobs[i]));
        }
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo

```
        removed++;
    }
}

if (!removed)
    printf("No completed jobs to remove.\n");
}

int main(void)
{
    char input[100];
    int number;
    char extra;

    printf("Q18 - Background Job Recovery\n");
    printf("Commands:\n");
    printf("add NUMBER\n");
    printf("stop JOB_ID\n");
    printf("bg JOB_ID\n");
    printf("jobs\n");
    printf("remove\n");
    printf("exit\n");

    while (1)
    {
        update_jobs();

        printf("\nrecovery> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "jobs") == 0)
        {
            list_jobs();
        }
        else if (strcmp(input, "remove") == 0)
        {
            remove_completed();
        }
        else if (sscanf(input, "add %d %c",
                        &number, &extra) == 1 &&
                 number >= 0)
        {
            start_job(number);
        }
        else if (sscanf(input, "stop %d %c",
                        &number, &extra) == 1)
        {
            stop_job(number);
        }
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 resume_jobs.c *
break;

input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "exit") == 0)
    break;

if (strcmp(input, "jobs") == 0)
{
    list_jobs();
}
else if (strcmp(input, "remove") == 0)
{
    remove_completed();
}
else if (sscanf(input, "add %d %c",
                &number, &extra) == 1 &&
         number >= 0)
{
    start_job(number);
}
else if (sscanf(input, "stop %d %c",
                &number, &extra) == 1)
{
    stop_job(number);
}
else if (sscanf(input, "bg %d %c",
                &number, &extra) == 1)
{
    resume_job(number);
}
else
{
    printf("Invalid command.\n");
}
}

/* Reap all remaining children before exiting */
for (int i = 0; i < MAX_JOBS; i++)
{
    if (jobs[i].state == STOPPED)
    {
        kill(-jobs[i].pgid, SIGCONT);
    }

    if (jobs[i].state == RUNNING ||
        jobs[i].state == STOPPED)
    {
        waitpid(jobs[i].pid, NULL, 0);
    }
}

printf("Background recovery program exited.\n");

return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo
```

```
return 0;
}

Write to File: resume_jobs.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel     M-M Mac Format  M-P Prepend    ^O Discard buffer
```

Save and compile:

gcc -Wall -Wextra resume_jobs.c -o resume_jobs

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano resume_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra resume_jobs.c -o resume_jobs
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 5 — Test Stopping and Resuming Jobs

Run:

```
./resume_jobs
```

Enter the following commands one at a time, without waiting too long between the first three:

```
add 60
```

```
jobs
```

```
stop 1
```

```
jobs
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./resume_jobs
Q18 - Background Job Recovery
Commands:
add NUMBER
stop JOB_ID
bg JOB_ID
jobs
remove
exit

recovery> add 60
Started job [1], PID=1041, PGID=1041

recovery> jobs

JOB TABLE
ID      PID      PGID      STATE      COMMAND
1       1041     1041     RUNNING    sleep 60

recovery> stop 1
Job [1] stopped.

recovery> jobs

JOB TABLE
ID      PID      PGID      STATE      COMMAND
1       1041     1041     STOPPED    sleep 60

recovery>
```

Now resume the job:

```
bg 1
```

Immediately enter:

```
jobs
```

After it completes, enter jobs again to display COMPLETED. Then enter:

remove

jobs

exit

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./resume_jobs
Q18 - Background Job Recovery
Commands:
add NUMBER
stop JOB_ID
bg JOB_ID
jobs
remove
exit

recovery> add 60
Started job [1], PID=1041, PGID=1041

recovery> jobs

JOB TABLE
ID      PID      PGID      STATE      COMMAND
1       1041     1041     RUNNING    sleep 60

recovery> stop 1
Job [1] stopped.

recovery> jobs

JOB TABLE
ID      PID      PGID      STATE      COMMAND
1       1041     1041     STOPPED    sleep 60

recovery> bg 1
SIGCONT sent to job [1].
Job resumed in background.

recovery> jobs

JOB TABLE
ID      PID      PGID      STATE      COMMAND
1       1041     1041     RUNNING    sleep 60

recovery> bg 1
Stopped job not found.

recovery> jobs

JOB TABLE
ID      PID      PGID      STATE      COMMAND
1       1041     1041     COMPLETED sleep 60

recovery> remove
Removing completed job [1].

recovery> jobs

JOB TABLE
ID      PID      PGID      STATE      COMMAND
No jobs found.

recovery> exit
Background recovery program exited.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```